

EE45: Intro to Foundations of ML

Mod 1 - Lec 4: Data Fitting Concepts Part IV

Overview: [Module 1] Data Fitting Concepts

- Last Week: Vector notation, norms products, loss functions from norms
- Today:
 - ▶ review k -means clustering, nearest neighbors, document dissimilarity
 - ▶ standardization of data

References:

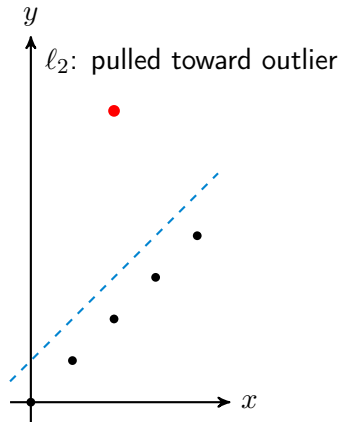
- [VMLS]: Chapters 1–3
- Topics: Vectors, Inner product, Linear functions, Regression model, Norm and distance

Content Summary

- Main Concepts

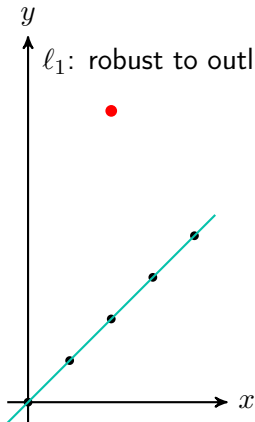
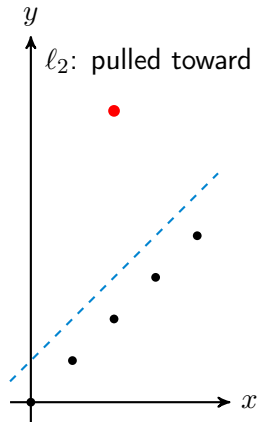


Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



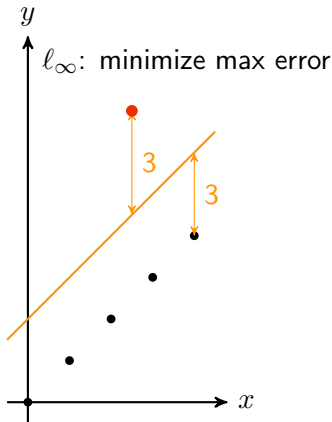
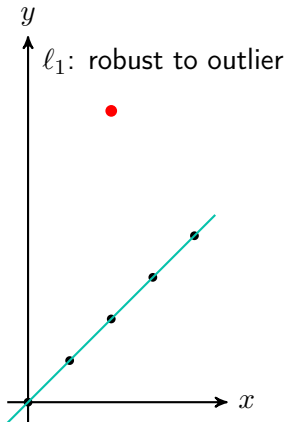
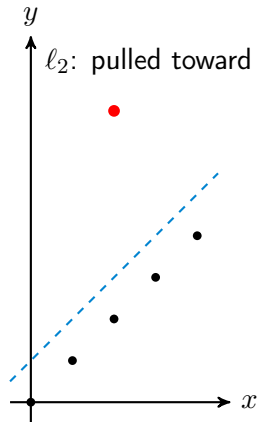
- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



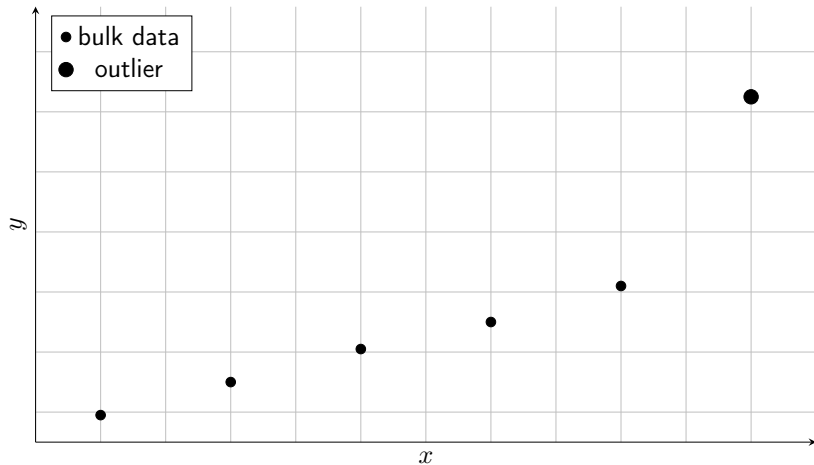
- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



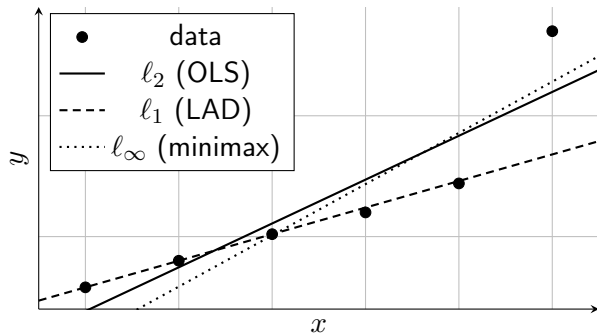
- ℓ_2 : Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1 : Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞ : *minimizes the largest absolute residual*

Choosing Norm Impacts Model Fit: Let's apply the logic



Note: Without this outlier, ℓ_2 and ℓ_1 give nearly identical fits. The outlier is the stress test.

Choosing Norm Impacts Model Fit



ℓ_2 : squares amplify the outlier $\Rightarrow$ slope tilts up.

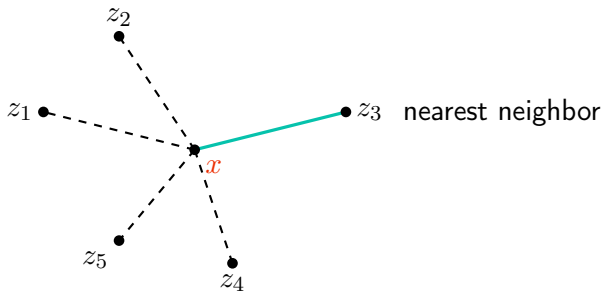
ℓ_1 : linear penalty $\Rightarrow$ stays near the bulk (robust).

ℓ_∞ : caps the worst error $\Rightarrow$ limits max deviation.

Feature distance and nearest neighbors

- if x, y are feature vectors for two entities, $\|x - y\|$ is the *feature distance*
- for vectors $z_1, \dots, z_m$, z_j is nearest neighbor of x if

$$\|x - z_j\| \leq \|x - z_i\|, \quad i = 1, \dots, m$$



Example: document dissimilarity

- Suppose we have m articles from Wikipedia: e.g., topics such as 'Veterans Day', 'Memorial Day', 'Academy Awards', 'Golden Globe Awards', 'Super Bowl'
- dictionary of n words: e.g.,
 $\text{dict} = \{\text{"in"}, \text{"world"}, \text{"red"}, \text{"blue"}, \text{"oscar"}, \text{"football"}, \dots\}$
- word counts of each article as "feature vectors": for each $i \in \{\text{'Veterans Day'}, \text{'Memorial Day'}, \text{'Academy Awards'}, \text{'Golden Globe Awards'}, \text{'Super Bowl'}, \dots\}$

$$x_i = \begin{bmatrix} \# \text{"in"} \\ \# \text{"world"} \\ \vdots \end{bmatrix}$$

- **Question:** How similar are two articles?

Example: document dissimilarity

	$i = 1$ Veterans' day	$i = 2$ Memorial day	$i = 3$ Academy A.	$i = 4$ Golden G.	$i = 5$ Super Bowl
Veterans' day	$\ x_1 - x_1\ $	$\ x_1 - x_2\ $	$\ x_1 - x_3\ $	$\ x_1 - x_4\ $	$\ x_1 - x_5\ $
Memorial day	$\ x_2 - x_1\ $	$\ x_2 - x_2\ $	$\ x_2 - x_3\ $	$\ x_2 - x_4\ $	$\ x_2 - x_5\ $
Academy A.	$\ x_3 - x_1\ $	$\ x_3 - x_2\ $	$\ x_3 - x_3\ $	$\ x_3 - x_4\ $	$\ x_3 - x_5\ $
Golden G.	$\ x_4 - x_1\ $	$\ x_4 - x_2\ $	$\ x_4 - x_3\ $	$\ x_4 - x_4\ $	$\ x_4 - x_5\ $
Super Bowl	$\ x_5 - x_1\ $	$\ x_5 - x_2\ $	$\ x_5 - x_3\ $	$\ x_5 - x_4\ $	$\ x_5 - x_5\ $

Example: document dissimilarity

- pairwise distances shown below:

	Veterans' day	Memorial day	Academy A.	Golden G.	Super Bowl
Veterans' day	0	0.095	0.130	0.153	0.170
Memorial day	0.095	0	0.122	0.147	0.181
Academy A.	0.130	0.122	0	0.108	0.164
Golden G.	0.153	0.147	0.108	0	0.164
Super Bowl	0.170	0.181	0.164	0.164	0

Document Dissimilarity

- We may want to cluster documents into groups.
- e..g, awards, sports, holidays, etc.
- We can use k-means to cluster documents into k groups.

What is K-Means Clustering?

- An unsupervised learning algorithm for clustering data.
- Goal: partition n observations into k clusters.
- Each observation belongs to the cluster with the nearest mean.

How Does It Work?

1. Choose the number of clusters k .
2. Initialize k cluster centroids (randomly)—i.e., $\mu_1, \dots, \mu_k \in \mathbb{R}^d$
3. **Assignment step:** Assign each data point $x_i \in \mathbb{R}^d$ to the nearest centroid.
4. **Update step:** Update centroids by taking the mean of assigned points.
5. Repeat until convergence (assignments don't change).

Key properties:

- Objective $\mathcal{L}$ decreases (or stays same) at each iteration
- Guaranteed to converge to a local minimum
- Result depends on initialization

Objective Function: Using Sum of Square Errors (Norms)

Within-Cluster Sum of Squares (WCSS)

$$\mathcal{L}(\{\mu_j\}_{j=1}^k, \{z_i\}_{i=1}^n) = \sum_{i=1}^n \sum_{x \in C_i} \|x - \mu_i\|^2$$

where C_i is cluster i and μ_i its centroid, and z_i is the cluster assignment for point x_i .

- k -means seeks to minimize $\mathcal{L}(\{\mu_j\}_{j=1}^k, \{z_i\}_{i=1}^n)$ with respect to μ_j for $j = 1, \dots, k$ and z_i for $i = 1, \dots, n$.

Centroid Update Rule

Given cluster assignments at iteration t , each centroid is updated to be the mean of the points in its cluster:

$$\mu_i^{(t+1)} = \frac{1}{|C_i^{(t)}|} \sum_{x \in C_i^{(t)}} x$$

- $C_i^{(t)}$: the set of points assigned to cluster i at iteration t .
- $|C_i^{(t)}|$: the number of points in that cluster.
- $\mu_i^{(t+1)}$: the new centroid, a vector in the same space $\mathbb{R}^n$.

Why the mean? The mean minimizes the sum of squared distances:

$$\mu_i^* = \operatorname{argmin}_{\mu} \sum_{x \in C_i} \|x - \mu\|^2$$

Minimizing a Function: First and Second Order Conditions

Problem: Find x^* that minimizes $f(x)$

First-order condition (necessary): $\left. \frac{df}{dx} \right|_{x=x^*} = 0$ The derivative is zero at critical points (minima, maxima, or saddle points).

Second-order condition (sufficient for minimum): $\left. \frac{d^2 f}{dx^2} \right|_{x=x^*} > 0$ Positive second derivative confirms x^* is a local minimum.

Multivariate case: For $f(\mathbf{x})$, require:

- $\nabla f(\mathbf{x}^*) = \mathbf{0}$ (gradient is zero)
- $\nabla^2 f(\mathbf{x}^*)$ is positive definite (Hessian)

Gradient and Hessian in Vector Notation

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a scalar-valued function of vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$.

Gradient: Vector of first partial derivatives $\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix} \in \mathbb{R}^d$

Hessian: Matrix of second partial derivatives

$$\nabla^2 f(\mathbf{x}) = H(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_d} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_d \partial x_1} & \frac{\partial^2 f}{\partial x_d \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_d^2} \end{bmatrix} \in \mathbb{R}^{d \times d}$$

Why the Mean Minimizes Sum of Squared Distances in K-Means

Objective: Minimize sum of squared distances to center μ

$$\mathcal{L}(\mu) = \sum_{i=1}^n \|\mathbf{x}_i - \mu\|_2^2 = \sum_{i=1}^n (\mathbf{x}_i - \mu)^\top (\mathbf{x}_i - \mu)$$

Expand:

$$\mathcal{L}(\mu) = \sum_{i=1}^n \left(\mathbf{x}_i^\top \mathbf{x}_i - 2\mathbf{x}_i^\top \mu + \mu^\top \mu \right)$$

Take derivative with respect to μ :

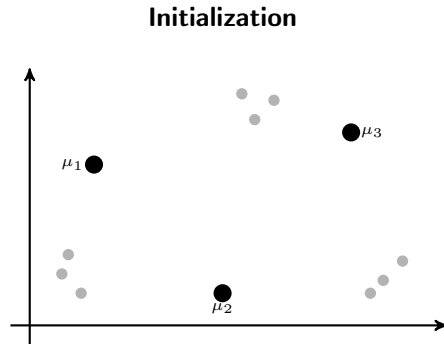
$$\frac{\partial \mathcal{L}}{\partial \mu} = \sum_{i=1}^n (0 - 2\mathbf{x}_i + 2\mu) = -2 \sum_{i=1}^n \mathbf{x}_i + 2n\mu$$

Set to zero and solve:

$$-2 \sum_{i=1}^n \mathbf{x}_i + 2n\mu = 0 \quad \Rightarrow \quad \mu^* = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$$

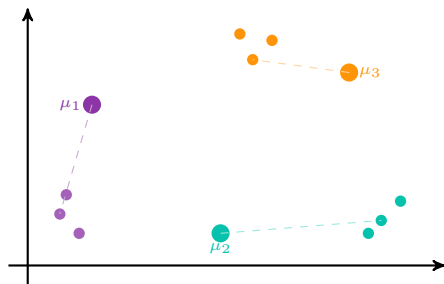
The optimal center is the mean of the cluster points.

K-Means Algorithm: Initialization



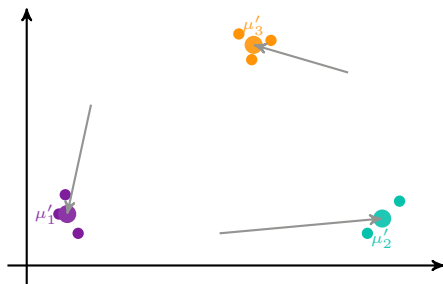
K-Means Algorithm: Iteration Steps

Iteration 1: Assign

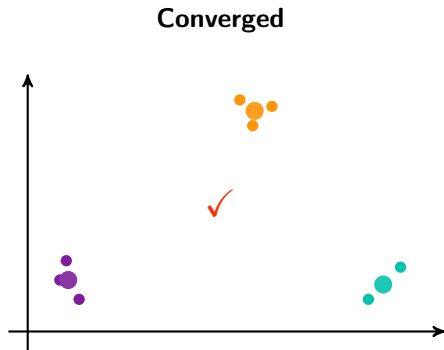


K-Means Algorithm: Iteration Steps

Iteration 1: Update



K-Means Algorithm: Iteration Steps



How Centroids Update: Computing the Mean

Update rule:

$$\mu_j^{\text{new}} = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

where $C_j = \{i : z_i = j\}$, points assigned to cluster j .

Example: Purple cluster has 3 points:

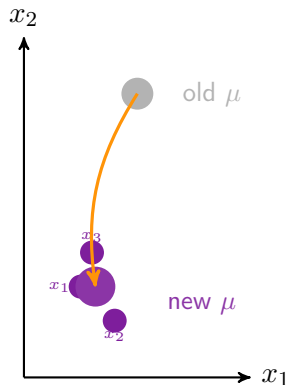
$$x_1 = (0.5, 0.8)$$

$$x_2 = (0.8, 0.5)$$

$$x_3 = (0.6, 1.1)$$

New centroid:

$$\mu_{\text{purple}} = \frac{1}{3}(x_1 + x_2 + x_3) = (0.63, 0.8)$$



The centroid moves to the **center of mass** of its assigned points.

K-Means Algorithm Animation

K-Means Algorithm

Goal. Partition data into k clusters by minimizing

$$J(\{\mu_j\}_{j=1}^k, \{z_i\}_{i=1}^n) = \sum_{i=1}^n \|x_i - \mu_{z_i}\|_2^2$$

where $z_i \in \{1, \dots, k\}$ is the cluster assignment for point x_i .

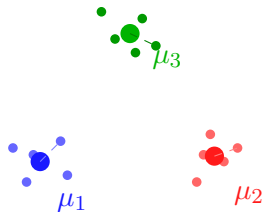
Algorithm (Lloyd's algorithm):

1. **Initialize** centers $\mu_1, \dots, \mu_k$ (randomly or via k-means++)
2. **Repeat** until convergence:
 - 2.1 **Assignment step:** For each point i , assign to nearest center:

$$z_i \leftarrow \operatorname{argmin}_{j \in \{1, \dots, k\}} \|x_i - \mu_j\|_2$$

- 2.2 **Update step:** For each cluster j , update center to mean:

$$\mu_j \leftarrow \frac{1}{|C_j|} \sum_{i \in C_j} x_i \quad \text{where} \quad C_j = \{i : z_i = j\}$$



K-Means with Different Norms

Norm	Distance metric	Optimal centroid	Cluster shape
ℓ_2	Euclidean	Mean	Round/spherical
ℓ_1	Manhattan	Componentwise median	Diamond-shaped
ℓ_∞	Chebyshev	Componentwise midrange	Square-shaped

Why ℓ_2 is standard:

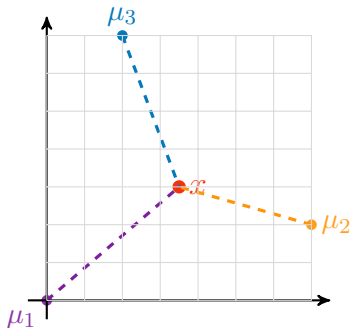
- Clean closed-form update: $\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$
- Computationally efficient (just averaging)
- Well-studied convergence properties
- Natural for many applications (physical distance, feature space)

But: ℓ_1 (k-medians) can be more robust to outliers!

Norms Can Disagree on Assignment

Centroids: $\mu_1 = (0, 0)$ (purple), $\mu_2 = (7, 2)$ (orange), $\mu_3 = (2, 7)$ (cerulean)

Test point: $x = (3.5, 3)$



Norm	$\ x - \mu_1\ $	$\ x - \mu_2\ $	$\ x - \mu_3\ $
ℓ_2	$\sqrt{12.25 + 9} = 4.61$	3.64	4.27
ℓ_1	$3.5 + 3 = 6.5$	4.5	5.5
ℓ_∞	$\max\{3.5, 3\} = 3.5$	3.5	4.0

- ℓ_2 : Smallest = 3.64 $\Rightarrow$ orange cluster (μ_2)
- ℓ_1 : Smallest = 4.5 $\Rightarrow$ orange cluster (μ_2)
- ℓ_∞ : TIE! (3.5 and 3.5)

Effect of Distance on K-Means

Euclidean (ℓ_2):

- Clusters are round (balls).
- Center = arithmetic mean.
- Sensitive to scale of features.

Manhattan (ℓ_1):

- Clusters look diamond-shaped.
- Center = componentwise median.
- More robust to outliers.

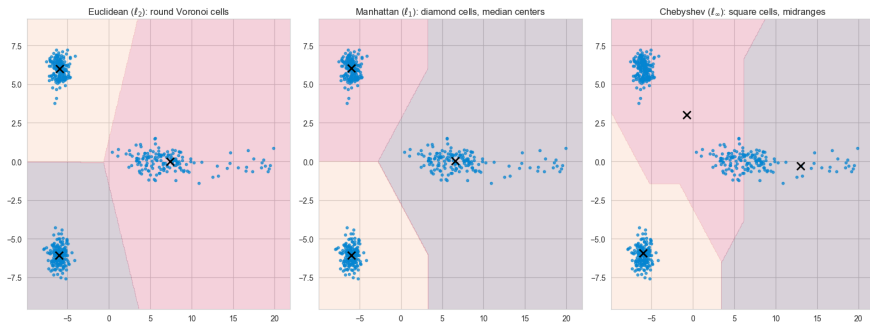
Cosine similarity:

- Clusters by angle, not magnitude.
- Center = normalized mean direction.
- Common for text or embeddings.

General lesson:

- Choice of distance = inductive bias.
- Different geometries $\Rightarrow$ different clusters.

K-Means with Different Distances



- ℓ_2 : round clusters, center on the elongated cluster is pulled toward the outliers (mean).
- ℓ_1 : diamond clusters, center sits closer to the componentwise median, resisting the long tail.
- ℓ_∞ : angular clusters, center sits at the midrange (avg of min/max per coordinate), so it shifts differently again.

How to Choose the Number of Clusters k ?

The challenge: k is a hyperparameter that must be specified before running k-means.

Common approaches:

1. **Domain knowledge:** Sometimes k is known from the problem
 - ▶ Customer segments, product categories, etc.
2. **Elbow method:** Plot objective $\mathcal{L}(k)$ vs. k , look for “elbow”
 - ▶ Most popular heuristic
 - ▶ Where adding more clusters gives diminishing returns
3. **Silhouette score:** Measure cluster quality
 - ▶ How well points fit their clusters vs. other clusters
4. **Gap statistic:** Compare to random data
 - ▶ More principled but computationally expensive

Reality: Often no single “correct” k ; depends on application.

The Elbow Method

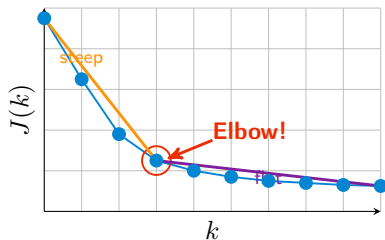
Idea: Plot $J(k)$ vs. k and look for the “elbow.”

Why does $J(k)$ decrease?

- More clusters $\Rightarrow$ points closer to centroids
- $J(1)$ = total variance
- $J(n) = 0$ (each point is its own cluster)

The elbow:

- Point where curve “bends”
- Diminishing returns after this
- Not always clear!



Choose $k = 4$ where marginal improvement flattens out.

Silhouette Score

Idea: Measure how well each point fits its cluster.

For each point i :

- $a(i)$ = average distance to other points in **same cluster**
- $b(i)$ = average distance to points in **nearest other cluster**

Silhouette coefficient:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

Interpretation:

- $s(i) \approx 1$: point well-clustered (far from other clusters)
- $s(i) \approx 0$: point on boundary between clusters
- $s(i) \approx -1$: point likely in wrong cluster

Average silhouette score: $\bar{s} = \frac{1}{n} \sum_{i=1}^n s(i)$

Choose k with highest $\bar{s}$.

Choosing k : Summary

Method	Pros	Cons
Elbow	Simple, intuitive, fast	Elbow not always clear; subjective
Silhouette	Quantitative measure of cluster quality	More computation; may favor compact clusters
Gap statistic	Principled comparison to null model	Expensive; requires many bootstrap samples
Domain knowledge	Most reliable when available	Not always available

Best practice:

- Try multiple methods
- Visualize results for different k
- Consider the application: what granularity makes sense?
- Remember: clustering is exploratory!

k-NN vs k-Means

k-Nearest Neighbors (k-NN)

- **Supervised learning algorithm**
- Used for classification (and regression)
- Predicts label of a new point by majority vote of its k nearest neighbors
- Relies on a distance metric (Euclidean, Manhattan, etc.)
- No training phase beyond storing the data

k-Means Clustering

- **Unsupervised learning algorithm**
- Used for clustering unlabeled data
- Iteratively assigns points to k clusters and updates cluster centers
- Objective: minimize within-cluster variance
- Training = alternating assignment and update until convergence

Both use k and a distance metric, but serve very different purposes.

k-Nearest Neighbors (k-NN)

Idea: Classify a new point based on the labels of its k nearest neighbors.

Key properties:

- **Supervised learning** algorithm (requires labeled training data)
- Used for both classification and regression
- **Non-parametric:** No explicit model/parameters to learn
- **Instance-based:** Stores entire training dataset
- **Lazy learning:** No training phase; all computation at prediction time

Decision rule (classification):

$$\hat{y} = \text{majority vote among } k \text{ nearest neighbors of } x$$

Example: Given a new email, is it spam or not spam? Look at the k most similar emails in your database and take a vote.

k-NN Classification: Visual Example

Setup:

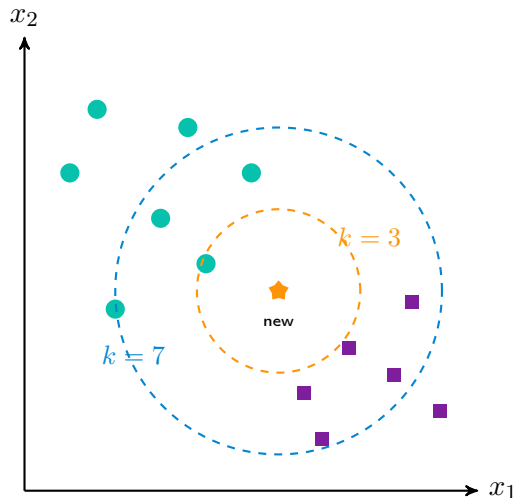
- Training data: red circles, blue squares
- New point: green star
- Question: What class?

$k = 3$:

- 3 nearest neighbors: 2 purple, 1 turquoise
- Prediction: **Purple**

$k = 7$:

- 7 nearest neighbors: 4 turquoise, 3 purple
- Prediction: **Turquoise**



The choice of k affects the prediction!

k-NN Algorithm

Training phase:

1. Store all training examples (x_i, y_i) for $i = 1, \dots, n$

That's it! (No optimization, no parameter fitting)

Prediction phase: Given a new point x^* ,

1. Compute distance from x^* to all training points:

$$d_i = \|x^* - x_i\| \quad \text{for } i = 1, \dots, n$$

2. Find the k points with smallest distances: $\mathcal{N}_k(x^*)$

3. **Output:**

[Classification] Return majority vote: $\hat{y} = \operatorname{argmax}_c \sum_{i \in \mathcal{N}_k(x^*)} \mathbf{1}[y_i = c]$

[Regression] Return average: $\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x^*)} y_i$

k-NN: Classification vs Regression

Same algorithm, different outputs depending on the task

Classification (discrete labels)

Goal: Predict category/class

Output: Label from finite set

- Spam/Ham
- Dog/Cat/Bird
- Digit 0-9

Decision rule:

$$\hat{y} = \text{mode}\{y_i : i \in \mathcal{N}_k(x)\}$$

(most frequent, majority vote among k neighbors)

Regression (continuous values)

Goal: Predict numeric value

Output: Real number

- House price
- Temperature
- Stock price

Decision rule:

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} y_i$$

(average of k neighbors)

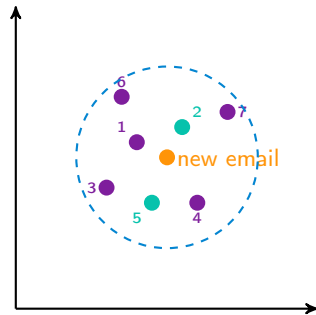
Let's Look at Examples

k-NN Classification: Understanding Majority Vote

Example: Email classification (Spam vs Ham) with $k = 7$

Step 1: Find 7 nearest neighbors

Neighbor	Distance	Label
1	0.12	Ham
2	0.18	Spam
3	0.21	Ham
4	0.25	Ham
5	0.29	Spam
6	0.31	Ham
7	0.35	Ham



Step 2: Count votes

- Ham: 5 votes; Spam: 2 votes

Prediction: Ham (majority wins)

Each neighbor gets **one vote** regardless of distance (unweighted k-NN)

Vote tally:

● ● ● ● ● Ham (5)

● ● Spam (2)

k-NN for Regression: Predicting Continuous Values

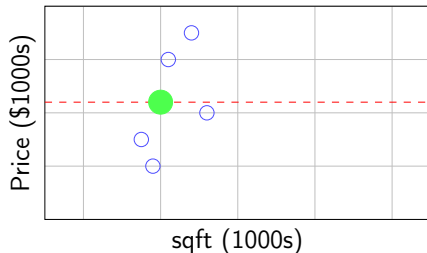
Example: Predict house price based on similar houses

New house: 1800 sqft, 3 bedrooms

- Price: ???

Find $k = 5$ nearest neighbors:

House	Distance	Price
A	0.08	\$250k
B	0.12	\$280k
C	0.15	\$240k
D	0.18	\$290k
E	0.22	\$260k



Red dashed line shows predicted price

Prediction (average): $\hat{y} = \frac{250 + 280 + 240 + 290 + 260}{5} = \$264k$

For regression: **average** the neighbor values, not vote!

Weighted k-NN: Distance Matters

Problem: All k neighbors vote equally, even if some are much farther away.

Solution: Weight votes by distance

Unweighted (standard):

$$\hat{y} = \operatorname{argmax}_c \sum_{i \in \mathcal{N}_k} \mathbf{1}[y_i = c]$$

All neighbors vote equally

Example: $k = 3$

- Neighbor 1 ($d=0.1$): Ham
- Neighbor 2 ($d=0.5$): Spam
- Neighbor 3 ($d=0.9$): Spam

Ham: 1, Spam: 2 $\Rightarrow$ **Spam**

Weighted:

$$\hat{y} = \operatorname{argmax}_c \sum_{i \in \mathcal{N}_k} w_i \cdot \mathbf{1}[y_i = c]$$

where $w_i = \frac{1}{d_i}$ or $w_i = \frac{1}{d_i^2}$

Same example: weights $w_i = 1/d_i$

- Ham: $1/0.1 = 10.0$
- Spam: $1/0.5 + 1/0.9 = 3.1$

Ham: 10, Spam: 3.1 $\Rightarrow$ **Ham**

Weighted k-NN gives closer neighbors more influence

Weighted K-NN: Distance as a Weighted Norm

Standard K-NN: Uses unweighted Euclidean distance

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_2 = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2}$$

Weighted K-NN: Uses weighted Euclidean distance

$$d(\mathbf{x}, \mathbf{x}_i) = \|\mathbf{x} - \mathbf{x}_i\|_W = \sqrt{\sum_{j=1}^d w_j (x_j - x_{ij})^2}$$

where $w_j > 0$ are feature weights.

Matrix form: Let $W = \text{diag}(w_1, w_2, \dots, w_d)$

$$d(\mathbf{x}, \mathbf{x}_i) = \sqrt{(\mathbf{x} - \mathbf{x}_i)^T W (\mathbf{x} - \mathbf{x}_i)}$$

This is a weighted ℓ_2 norm: $\|\mathbf{v}\|_W = \sqrt{\mathbf{v}^T W \mathbf{v}}$

Handling Ties in k-NN

Problem: What if neighbors vote equally for multiple classes?

Strategies for breaking ties:

1. **Use odd k** (for binary classification)
 - ▶ Guarantees no 50-50 split
 - ▶ But can still tie with 3+ classes
2. **Pick the class of the nearest neighbor**
 - ▶ Among tied classes, choose the one with smallest min distance
3. **Random choice**
 - ▶ Randomly select among tied classes
4. **Use weighted voting**
 - ▶ Weight by $1/d_i \rightarrow$ ties are rare
5. **Reduce k**
 - ▶ Use $k - 1$ neighbors instead

Best practice: Use odd k and/or weighted voting

Choosing k in k-NN

Important: k is the number of neighbors to consider for prediction.

Small k (e.g., $k = 1$):

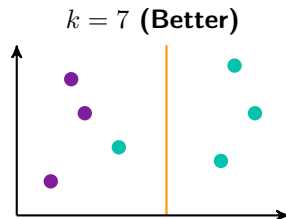
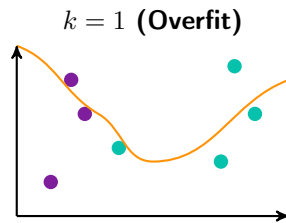
- Very flexible decision boundary $\implies$ Can overfit
- Low bias, high variance $\implies$ Sensitive to noise

Large k :

- Smoother decision boundary $\implies$ Can underfit
- High bias, low variance $\implies$ More robust to noise

Typical choice: $k = \sqrt{n}$ (rule of thumb)

- Use cross-validation
- Odd k for binary classification (avoid ties)



Choosing Distance Metrics for k-NN

Metric	Formula	When to use
Euclidean (ℓ_2)	$\sqrt{\sum_j (x_j - x'_j)^2}$	• Continuous features w/ similar units • Most common default
Manhattan (ℓ_1)	$\sum_j x_j - x'_j $	• Grid-like (city blocks) or High-dimensional data • More robust to outliers
Cosine	$1 - \frac{x \cdot x'}{\ x\ \ x'\ }$	• Text data (word counts) • magnitude doesn't matter & direction more important
Hamming	$\sum_j \mathbf{1}[x_j \neq x'_j]$	• Categorical features • Binary/discrete data (e.g., DNA sequences)

Always normalize/standardize features first!

Distance Metrics in k-NN

The choice of distance metric affects which points are "nearest."

Common distance metrics:

Metric	Formula	When to use
Euclidean (ℓ_2)	$\sqrt{\sum_{j=1}^d (x_j - x'_j)^2}$	General purpose; continuous features
Manhattan (ℓ_1)	$\sum_{j=1}^d x_j - x'_j $	Grid-like spaces; robust to outliers
Chebyshev (ℓ_∞)	$\max_j x_j - x'_j $	When all features matter equally
Cosine	$1 - \frac{x \cdot x'}{\ x\ \ x'\ }$	Text, high-dim sparse data

Important: Features should be on similar scales!

- Feature with large range will dominate distance
- Solution: **standardize features** before applying k-NN

Why Feature Scaling Matters for k-NN

Example: Predicting house price using area and # bedrooms

Without scaling:

Training data:

- House A: (2000 sqft, 3 beds)
- House B: (1500 sqft, 2 beds)

New house: (1900 sqft, 2 beds)

Distance to A:

$$\begin{aligned}d &= \sqrt{(1900 - 2000)^2 + (2 - 3)^2} \\ &= \sqrt{100^2 + 1^2} \approx 100\end{aligned}$$

Area dominates! Bedrooms barely matter.

With scaling:

Standardized:

- House A: (1.2, 0.8)
- House B: (-0.6, -0.4)

New house: (0.9, -0.4)

Distance to A:

$$\begin{aligned}d &= \sqrt{(0.9 - 1.2)^2 + (-0.4 - 0.8)^2} \\ &= \sqrt{0.09 + 1.44} \approx 1.24\end{aligned}$$

Both features contribute equally.

Always standardize features for k-NN!

k-NN vs k-Means: Key Differences

	k-NN	k-Means
Type	Supervised (needs labels)	Unsupervised (no labels)
Task	Classification/Regression	Clustering
What is k?	Number of neighbors to consider	Number of clusters
Training	None (stores data)	Iterative optimization
Prediction	Expensive (compute all distances)	Fast (assign to nearest centroid)
Output	Label or value for new point	Cluster assignments for all points
Memory	High (stores all training data)	Low (only stores k centroids)

Both:

- Use distance metrics (Euclidean, Manhattan, etc.)
- Sensitive to feature scaling
- Require choosing k (but means different things!)

Example: Document Classification with k-NN

Task: Classify a new Wikipedia article by topic

Setup:

- Feature vector: word count histogram ($n = 4423$ words)
- Distance: Euclidean distance on normalized counts
- $k = 3$ nearest neighbors

Training data:

- Veterans Day → Holiday
- Memorial Day → Holiday
- Academy Awards → Award Show
- Golden Globes → Award Show
- Super Bowl → Sports

New article: “Labor Day”

1. Compute distances to all 5 training articles
2. 3 nearest: Veterans Day (0.08), Memorial Day (0.10), Academy Awards (0.15)
3. Majority vote: 2 Holiday, 1 Award
4. **Prediction:** Holiday ✓

Document Classification: k-NN vs k-Means

Example corpus:

Veterans Day, Memorial Day, Academy Awards, Golden Globes, Super Bowl

k-Means (unsupervised)

- Input: pairwise document distances (from word-count histograms).
- Goal: cluster into $k = 3$ groups.
- Outcome:
 - ▶ Veterans Day, Memorial Day → Holidays
 - ▶ Academy Awards, Golden Globes → Awards
 - ▶ Super Bowl → Sports
- No labels required.

k-NN (supervised)

- Assume we already have labels: Holiday, Award Show, Sports.
- New document: “Labor Day.”
- Compute distances to all 5 documents.
- Nearest neighbors (Veterans Day, Memorial Day).
- Majority label = Holiday $\Rightarrow$ classify as Holiday.

Same distance representation, two different uses.

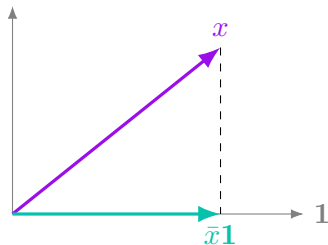
(If time permits) Standardizing Data

Example: Mean as Inner Product

- The mean is the "balance point" (center of mass).
- The sample mean is

$$\mu = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{\langle x, \mathbf{1} \rangle}{\langle \mathbf{1}, \mathbf{1} \rangle}$$

- Geometrically: $\bar{x} \cdot \mathbf{1}$ is the **projection** of x onto the "span" of $\mathbf{1}$
- Intuition: the mean is the "best constant approximation" of the data

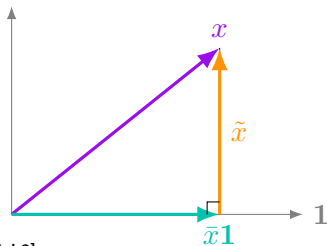


Centering Feature Vectors

Given $x \in \mathbb{R}^n$ and $\mathbf{1} = (1, 1, \dots, 1)^\top \in \mathbb{R}^n$:

$$\tilde{x} = x - \bar{x} \mathbf{1} = x - \frac{1}{n} \mathbf{1}^\top x \mathbf{1} = x - \frac{1}{n} \mathbf{1} \mathbf{1}^\top x = \left(I - \frac{1}{n} \mathbf{1} \mathbf{1}^\top \right) x$$

- $\mathbf{1} \mathbf{1}^\top \in \mathbb{R}^{n \times n}$ is the all-ones matrix.
- $H = I - \frac{1}{n} \mathbf{1} \mathbf{1}^\top$ is the **centering matrix**.
- H is symmetric and idempotent ($H^2 = H$): an **orthogonal projection**.
- Geometrically: projects x onto the subspace orthogonal to $\mathbf{1}$.



Example: Variance and Standard Deviation

- The centered data vector is $x - \mu \mathbf{1}$
- The standard deviation is the "typical distance" of points from that balance point.
 - ▶ Small SD $\rightarrow$ data clustered tightly around the mean.
 - ▶ Large SD $\rightarrow$ data spread out.
- If you pick a random data point, the standard deviation is roughly the size of the "typical error" you'd make if you always guessed the mean.
- Think of the mean as a reference position. The standard deviation is like the moment of inertia—how far mass is spread out from the center. Same "center," but very different spread.
- The variance is the **scaled average squared deviation**

Example: Variance and Standard Deviation

- The centered data vector is $\tilde{x} = x - \bar{x}\mathbf{1} = Hx$, where $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is the centering matrix. H projects onto the subspace orthogonal to $\mathbf{1}$
- Its squared norm gives the total squared deviation:

$$\|x - \mu\mathbf{1}\|^2 = \sum_{i=1}^n (x_i - \mu)^2 = \|\tilde{x}\|^2 = \|Hx\|^2$$

- The variance is the **scaled average squared deviation**:

$$\text{Var}(x) = \sigma^2 = \frac{1}{n}\|x - \bar{x}\mathbf{1}\|^2 = \frac{1}{n}\|\tilde{x}\|^2 = \frac{1}{n}\|Hx\|^2$$

- Standard deviation:

$$\text{Std}(x) = \sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} = \sqrt{\frac{1}{n}\|Hx\|^2} = \frac{1}{\sqrt{n}} \|Hx\|.$$

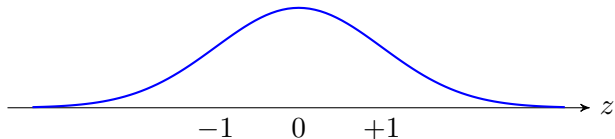
Standardizing Data

Given data $x = (x_1, \dots, x_n)$ with mean $\bar{x}$ and std. deviation s , the **standardized data** (z-scores) are

$$z_i = \frac{x_i - \bar{x}}{s}.$$

Properties

- Centered: $\text{mean}(z) = 0$
- Scaled: $\text{sd}(z) = 1$
- Interpretable in **standard deviation units**.
- Comparable across different variables



Standardizing Data (z-scores)

Given $x \in \mathbb{R}^n$, mean $\bar{x}$, and centering matrix

$$H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top,$$

the **centered data** is

$$\tilde{x} = Hx.$$

The **standard deviation** is

$$\text{std}(x) = \frac{1}{\sqrt{n}}\|Hx\|.$$

Standardized data (z-scores):

$$z = \frac{\tilde{x}}{\text{std}(x)} = \frac{Hx}{\frac{1}{\sqrt{n}}\|Hx\|} = \sqrt{n} \frac{Hx}{\|Hx\|}.$$

Standardizing Data = z-scores

Statistics: z-scores

$$z = \frac{x - \mu}{\sigma}$$

- Measures how many **standard deviations** a point x is from the mean.
- Example: $x = 70$, $\mu = 50$, $\sigma = 10$
 $\Rightarrow z = 2$.
- Used to compare values on a common scale and link to the standard normal.

ML: feature standardization

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

- Compute a z-score for **each feature** j across samples.
- Ensures all features have mean 0 and std 1.
- Important for distance-based models (kNN, SVM) and optimization (GD, PCA).

In ML, "standardizing data" = computing z-scores feature-wise.

Why (and When Not) to Standardize Data

- **Why standardize:**
 - ▶ **Comparability across features** Different units (kg, \$, counts) → put in common **std dev units**.
 - ▶ **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
 - ▶ **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
 - ▶ **Interpretability** Coefficients are comparable across features.

Why (and When Not) to Standardize Data

- **Why standardize:**

- ▶ **Comparability across features** Different units (kg, \$, counts) → put in common **std dev units**.
- ▶ **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
- ▶ **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
- ▶ **Interpretability** Coefficients are comparable across features.

- **When **not** to standardize:**

- ▶ **Categorical features** encoded as one-hot or integers → standardizing breaks their meaning.
- ▶ **Features with physical meaning in the model** e.g. age in years, probability values, counts with direct interpretability.
- ▶ **Tree-based models** (decision trees, random forests, gradient boosting) → splits are order-based, not distance-based; scaling irrelevant.